Binary Files

03 August 2024 18:28

Binary file handling in Java involves reading from and writing two files in binary format. This is useful when working with non-text data, such as images, audio files or serialized objects.
Java provides several classes in the **"java.io"** package, to handle binary files including 'FileInputStream', 'FileOutputStream', 'DataInputStream', 'DataOutputStream' and 'BufferedInputStream' and 'BufferedOutputStream'.

Key Concepts

1. **FileInputStream and FileOutputStream:** These classes are used to read and write streams of raw bytes.
2. **DataInputStream and DataOutputStream:** These classes allow you to read and write Java primitive data types. [int, char, float, etc.] in a machine-independent way.
3. **BufferedInputStream and BufferedOutputStream:** These classes provide buffer for InputStream and OutputStream respectively for more efficient reading and writing.

The try-with-resources Statement

The try-with-resources statement is a try statement that declares one or more resources. A *resource* is an object that must be closed after the program is finished with it. The try-with-resources statement ensures that each resource is closed at the end of the statement. Any object that implements java.lang.AutoCloseable, which includes all objects which implement java.io.Closeable, can be used as a resource.

interface.

interface.

https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html

```
package FileHandling1.BinaryFiles;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Scanner;
public class BinaryFileWriter {
    public static void main(String[] args) {
        String fileName;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        fileName = sc.nextLine();
        try(FileOutputStream fos = new FileOutputStream("./DataFiles/"+fileName)){
            fos.write(100);
            fos.write(new byte[]{65, 66, 67});
            sc.close();
        }
        catch(FileNotFoundException e){}
        catch(IOException e){}
    }
}
```

DataOutputStream

```
package FileHandling1.BinaryFiles;
import java.io.DataOutputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Scanner;
```

```java
public class BinaryFileWriterDOS {
    public static void main(String[] args) {
        String fileName;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        fileName = sc.nextLine();
        sc.close();
        try(DataOutputStream dos = new
        DataOutputStream(new FileOutputStream("./DataFiles/"+fileName))){
            dos.writeInt(123);
            dos.writeDouble(45.67);
            dos.writeBoolean(false);
            dos.writeUTF("Java programming is fun");
        }
        catch(FileNotFoundException e){}
        catch(IOException e){}
    }

}
```

BufferedOutputStream

```java
package FileHandling1.BinaryFiles;
import java.io.BufferedOutputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Scanner;
public class BufferedBinaryFileWriter {
    public static void main(String[] args) {
        String fileName;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        fileName = sc.nextLine();
        sc.close();
        try (BufferedOutputStream bos = new
        BufferedOutputStream(new FileOutputStream("./DataFiles/" + fileName))) {
            bos.write(5672);
            bos.write(new byte[]{65,66,67,68});
        } catch (FileNotFoundException e) {
            // TODO: handle exception
        }
        catch(IOException e){}
    }
}
```

Reading from a binary file

1. **Using FileInputStream**

```java
package FileHandling1.BinaryFiles;
import java.io.FileInputStream;
import java.util.Scanner;
public class BinaryFileReader {
    public static void main(String[] args) {
        String fileName;
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Enter file name: ");
        fileName = sc.nextLine();
        sc.close();
        try (FileInputStream fis = new FileInputStream("./DataFiles/"+fileName)) {
            int data;
            while((data = fis.read())!=-1){
                System.out.print(data);
            }
        } catch (Exception e) {
            // TODO: handle exception
        }
    }
}
```

2 Using DataInputStream

```
package FileHandling1.BinaryFiles;
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.InputStream;
import java.util.Scanner;
public class BinaryFileReaderDIS {
    public static void main(String[] args) {
        String fileName;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        fileName = sc.nextLine();
        sc.close();
        try (DataInputStream dis = new DataInputStream(new
            FileInputStream("./DataFiles/"+fileName))) {
            int ivalue = dis.readInt();
            double dvalue = dis.readDouble();
            boolean bvalue = dis.readBoolean();
            String svalue = dis.readUTF();
            System.out.println("\nInteger: "+ivalue+
            "\nDouble value: "+dvalue+
            "\nBoolean value: "+bvalue+
            "\nString value: "+svalue);
        } catch (Exception e) {
            // TODO: handle exception
        }
    }
}
```

3. Using BufferedInputStream

```
package FileHandling1.BinaryFiles;
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Scanner;
public class BufferedBinaryFileReader {
    public static void main(String[] args) {
        String fileName;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        fileName = sc.nextLine();
```

```
            sc.close();
            try (BufferedInputStream bis = new BufferedInputStream(
                new FileInputStream("./DataFiles/"+fileName) )) {
                int data;
                while((data = bis.read()) != -1){
                    System.out.println((char)data);
                }
            }
            catch(FileNotFoundException e){}
            catch(IOException e){}
        }
    }
```

WAP to create a pattern by entering number of lines and store the patten by entering the file name. For example:
Input: Enter number of lines(2<line<10): 6
Output:

```
      6
     5 6 5
    4 5 6 5 4
   3 4 5 6 5 4 3
  2 3 4 5 6 5 4 3 2
 1 2 3 4 5 6 5 4 3 2 1
```

Enter file name: six.txt

R
I
S
H
I

Homework:
WAP to input a name and print the name in the following format:

Input: Enter your name: RISHI
Output:

```
********    ********    ********    *       *    ********
*       *       **      *           *       *       **
*       *       **      *           *       *       **
******          **      ********    ********        **
*       *       **              *   *       *       **
*       *       **              *   *       *       **
*       *    ********   ********    *       *    ********
```

use string formatting commands to print the characters properly

26 files in folder

R.txt I.txt S.txt

```
********    ********    ********    *       *    ********
*       *       **      *           *       *       **
*       *       **      *           *       *       **
******          **      ********    ********        **
*       *       **              *   *       *       **
*       *       **              *   *       *       **
*       *    ********   ********    *       *    ********
```